
COMP90024 CLUSTER & CLOUD COMPUTING ASSIGNMENT 1

LANGUAGE-COUNTING PROGRAMS

LINZHONG GUO
1857963

Muhammad Rayes Fitra Gifari
1848016

April 14, 2026

ABSTRACT

This report presents a parallel solution for counting language usage in large Mastodon and BlueSky NDJSON datasets on the University of Melbourne Spartan HPC system. Two MPI-based strategies, modulo-based assignment and byte-chunking, were implemented and compared under different runtime configurations. The results show that byte-chunking achieved the best performance, with the fastest runtime of 170.9 seconds using 1 node and 8 cores. Amdahl's Law was applied to estimate the theoretical speedup limit, showing that performance is constrained by the remaining serial portion of the workflow.

Keywords: MPI, Spartan HPC, parallel computing, byte-chunking, NDJSON, language counting

1 Introduction

This project develops parallel language-counting programs for large-scale Mastodon and BlueSky datasets on the Spartan HPC system. The datasets are stored in NDJSON format and include small, medium, and large files, with the large BlueSky and Mastodon datasets reaching 121 GB and 91 GB respectively. The report focuses on how language information is extracted, how the computation is parallelised with MPI, and how runtime performance changes under different resource configurations. Two parallelisation methods, modulo-based assignment and byte-chunking, are evaluated and compared. The results show that byte-chunking provides the best overall performance and is therefore selected as the final implementation.

2 Dataset and Preparation

2.1 Dataset overview

The assignment uses Mastodon and BlueSky datasets stored in NDJSON format. Each line of the file represents a single JSON object. Small and medium datasets were used for local testing and debugging, while the large datasets were executed on Spartan HPC for final performance evaluation.

2.2 Assumptions on language extraction

The datasets are stored in NDJSON format, where each line represents one complete JSON object. For Mastodon, language information is extracted from `doc.language`, while for BlueSky it is extracted from `record.langs`. Mastodon records contain a single language tag, whereas BlueSky records may contain multiple tags, each of which is counted separately if non-empty. All language tags are normalised to lowercase, while regional tags such as `en-au` and `zh-cn` are preserved. Records with missing language information are counted as “No language”, and malformed records are counted as “Bad JSON”.

Example	Description
Mastodon: {"doc": {"language": "en"}}	en is extracted and counted
Mastodon: {"doc": {"language": null}}	treated as "No language"
BlueSky: {"record": {"langs": ["en", "fr"]}}	en and fr are both counted
BlueSky: {"record": {"langs": ["EN-AU", "zh-cn"]}}	converted to en-au and zh-cn, then counted separately
BlueSky: {"record": {"langs": []}}	treated as "No language"
Invalid JSON line	treated as "Bad JSON" and excluded

Table 1. Examples of language extraction rules

2.3 Preparation and structure inspection

Before implementation, both datasets were inspected manually and with `checkdata.py` to confirm the location and structure of the language fields. This preparation was necessary because the language field is not located at the top level of the JSON object, and each platform uses a different nested structure. Understanding the nested structure of each dataset allows the program to correctly extract valid language values and apply the subsequent parallel counting process.

3 Implementation

3.1 First iteration: pandas baseline

A pandas-based implementation was first used as a baseline to validate language extraction and counting logic on small datasets. While it produced correct outputs, it was not suitable for large-scale execution because it requires loading and processing the dataset in a less scalable way. This limitation motivated the transition to an MPI-based solution.

3.2 Second iteration: MPI counting

The second stage introduced MPI-based parallel counting using `mpi4py`. Each rank processes a subset of the input data, maintains a local Counter, and reports the result to the root rank for aggregation. In addition to language counts, the program also records malformed JSON entries and missing-language cases. This stage established the core parallel workflow and produced correct outputs on small and medium datasets.

3.3 Two parallelisation strategies

Two parallelisation strategies were then implemented and compared. In the modulo-based method, each rank processes records according to its assigned index pattern. In the byte-chunking method, the input file is divided into byte ranges, and each rank processes only its allocated chunk (Adefemi, 2016). For non-root ranks, the first partial line at the chunk boundary is skipped to avoid duplicated or broken records. Compared with the modulo-based approach, byte-chunking reduces unnecessary reading and improves workload distribution for very large files.

4 Results and Discussion

4.1 Experimental setup

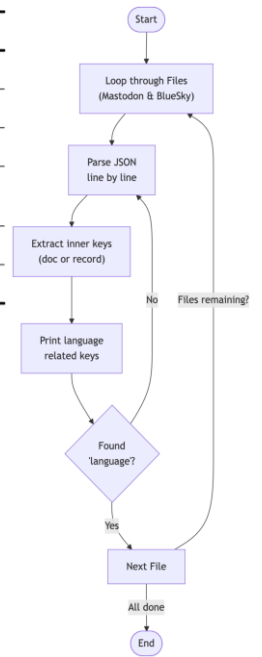


Figure 1. Language field inspection workflow

The large datasets were evaluated on Spartan HPC under three configurations: 1 node 1 core (1n1c), 1 node 8 cores (1n8c), and 2 nodes 8 cores (2n8c). These configurations were used consistently to compare the runtime behaviour of the modulo-based and byte-chunking methods.

4.2 Method 1: modulo-based parallelisation

The modulo-based implementation produced identical top 10 language counts and missing-language totals across all three configurations, indicating that the method was functionally stable. However, its runtime performance was limited because each rank still depended on a less efficient access pattern across the dataset. Although parallel execution improved runtime compared with the single-core baseline, the gains were smaller than those achieved by byte-chunking.

Configuration Runtime (s) Speedup

1n1c	1771.62	1.00
1n8c	548.25	3.23
2n8c	511.99	3.46

Table 2. Runtime and speedup

4.3 Method 2: byte-chunking parallelisation

The byte-chunking method achieved substantially lower runtime by assigning each rank a dedicated byte range of the input file. This reduced redundant reading and enabled more balanced workload distribution across ranks. The root rank was responsible for aggregating local counts and printing the final output, while the worker ranks focused only on data processing. Under this design, the 1n8c configuration delivered the best overall performance, while 2n8c showed no clear additional benefit due to inter-node communication overhead.

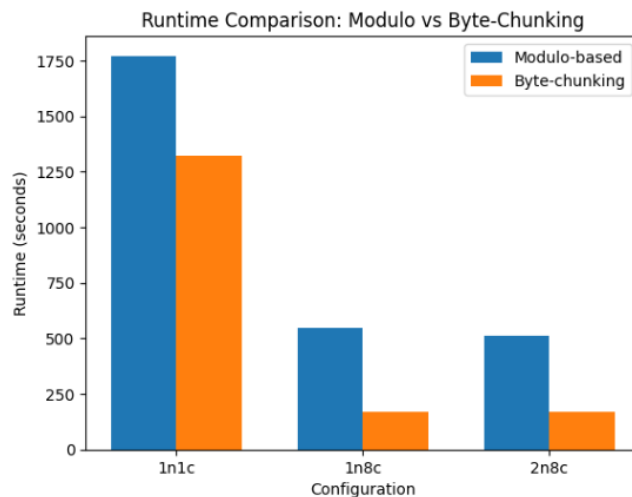


Figure 2. Runtime comparison

As shown in Figure 1, the byte-chunking method consistently outperforms the modulo-based approach across all configurations. The performance improvement is particularly significant in parallel settings, where chunking enables more efficient workload distribution and reduces redundant file access. Overall, both methods produced consistent counting results, but byte-chunking was more efficient in runtime. The main advantage of byte-chunking is that it partitions the file directly by byte offsets, allowing each rank to process a distinct portion of the dataset with less overhead. For this reason, byte-chunking was selected as the final method.

Bluesky - Bad JSON: 0 | No language: 641,722
Mastodon - Bad JSON: 0 | No language: 644,801

Time taken: 1319.8798 seconds

#	Mastodon Languages Used	Frequency of Occurrence (#posts)	BlueSky Languages Used	Frequency of Occurrence (#posts)
1	En (English)	22,492,294	En (English)	72,772,745
2	De (German)	1,635,575	De (German)	2,103,551
3	Es (Spanish)	385,703	Es (Spanish)	635,915
4	Fr (French)	383,728	Fr (French)	482,958
5	Zh (Chinese)	289,853	Pt (Portuguese)	269,539
6	Nl (Dutch)	221,422	Pl (Polish)	228,583
7	Ru (Russian)	189,917	Ja (Japanese)	227,321
8	Ja (Japanese)	178,651	Fi (Finnish)	143,567
9	It (Italian)	157,979	En-Au (English)	120,529
10	Fi (Finnish)	131,992	Nl (Dutch)	109,243

Bluesky - Bad JSON: 0 | No language: 641,722
Mastodon - Bad JSON: 0 | No language: 644,801

Time taken: 170.9015 seconds

Bluesky - Bad JSON: 0 | No language: 641,722
Mastodon - Bad JSON: 0 | No language: 644,801

Time taken: 171.9516 seconds

Figure 3. Top 10 language counts by the byte-chunking

5 Amdahl's Law Discussion of the Final Method

The processing times for the final method are 1319.88 s for 1n1c, 170.90 s for 1n8c, and 171.95 s for 2n8c. To calculate the actual speedup, the serial runtime is divided by the runtime of the parallel execution using 8 cores.

$$S(N) = \frac{1319.88}{170.9} = 7,723$$

Next, Amdahl's Law is applied to estimate the parallelisable portion of the program.

$$S = \frac{1}{(1-P) + \frac{P}{n}} \Rightarrow \frac{1}{S} = (1-\alpha) + \frac{\alpha}{N} \Rightarrow \frac{1}{7,723} = (1-\alpha) + \frac{1}{8}\alpha \Rightarrow 0.87 = 0.875\alpha$$

$$\alpha = \frac{0.87}{0.875} = 0.9942$$

Based on the calculation, 99.42% of the program can be parallelised. The theoretical maximum speedup can then be calculated as follows.

$$\frac{1}{1-\alpha} = \frac{1}{1-0.9942} = \frac{1}{0.0058} = 172.41$$

The theoretical maximum speedup is 172.41×. Therefore, the minimum theoretical runtime under ideal conditions is 1319.88 / 172.41 = 7.65 seconds. According to Amdahl's Law, this represents the lower bound of execution time for processing the large BlueSky and Mastodon datasets. Even if more computational resources are added, the runtime cannot be reduced beyond this limit because some parts of the program remain inherently serial, such as file input/output and result aggregation.

6 Conclusion

This project implemented and evaluated two MPI-based parallelisation strategies for language counting on large Mastodon and BlueSky datasets. Both methods produced stable and correct counting results across different runtime configurations. However, byte-chunking achieved substantially better runtime performance because it reduced redundant file access and distributed the workload more efficiently. Among the tested configurations, 1n8c delivered the best overall result, while 2n8c did not improve further due to additional inter-node communication overhead. Therefore, byte-chunking with 1n8c is the most effective configuration for this task on Spartan HPC.

AI Usage

Claude and ChatGPT were used only for limited support in summarising and organising information during the development of this project. All core technical work, including understanding the dataset structure, designing the parallelisation methods, implementing the code, conducting experiments, interpreting the results, and writing the final report, was completed independently by the authors. AI tools were not used to generate the main technical solution or determine the project methodology.

References

Adefemi, T. (2016). *What Every Computer Scientist Needs to Know About Parallelization*. Arxiv.org. <https://arxiv.org/html/2504.03647v1>

Ali, S. (2025, February 13). *Why You Should Avoid Pandas in Production Applications*. DEV Community. <https://dev.to/sajjadali/why-you-should-avoid-pandas-in-production-applications-2n3i>